Portfolio Optimization with Python - Case Study

Instructions

In this case study, we will build two $10,000 investment portfolios containing four stocks. The first portfolio will have an equal weighting between the stocks. The second portfolio will be optimized with a weighting allocation that provides the best return, adjusted for risk.

Set Up

Import two years of data for four stocks

Import the following packages for the case study:

- Pandas
- Numpy
- Matplotlib.pyplot

Create a list named **stock_list** that contains the ticker symbols for the four stocks in the portfolio:

- AMD
- AAPL
- MSFT
- ORCL

Create an empty dictionary, **stocks,** to store the stock information and create a for loop to import the associated .csv file for each ticker from **stock_list**.

Equal Weighted Portfolio

Create the Equal-Weighted Portfolio

To create the equal-weighted portfolio, we need to add three new columns to the DataFrames in the **stocks** dictionary:

- Normalized Return — Adjusted Close / Adjusted Close on the `startdate` of the portfolio
- Allocation — Normalized Return * 0.25
- Position Value — Allocation * $10,000

Visualize the Portfolio Performance

Build a new DataFrame **position_values** that contains the `position value` for each stock, as well as the total value for the portfolio.

To visualize the performance of the portfolio, create two line charts that show the return of the portfolio, and the return of the individual stocks, over time.

Calculate Performance Metrics for the Portfolio

Calculate five performances metrics:

- Cumulative Return — End Value of Portfolio / Start Value of Portfolio
- Mean Daily Return — Mean of daily % change in Portfolio value
- Standard Deviation Daily Return — Std Dev of daily % change in Portfolio value
- Sharpe Ratio — Mean Daily Return / Std Dev Daily Return
- Annualized Sharpe Ratio — Sharpe Ratio ** Sqrt of Number of Trading Days

Sharpe Ratio

The Sharpe ratio is calculated by the average return of the portfolio minus a risk free rate (such as government bonds), divided by the standard deviation of the return. In this case, we assume the risk-free rate is close to zero and 252 trading days in a year.

Optimized Portfolio

Prepare Scenarios to Optimize Portfolio Weighting

Create a dictionary **stock_adj_close** containing the adjusted close for each of the stocks. Create another dictionary **stock_returns** that transforms the adjusted close for each day to a percent change from the previous day.

Build & Run 10,000 Portfolio Scenarios

Build the structures required to generate these scenarios and store the output using **numpy.zeros()**:

- **weights_array** contains the weighting allocation for each stock
- **returns_array** contains the portfolio return for each scenario
- **volatility_array** contains the portfolio volatility for each scenario
- **sharpe_array** contains the Sharpe Ratio for each scenario

Identify the Optimal Portfolio

Now that we have the output for all 10,000 scenarios, identify the optimal portfolio. The optimal portfolio in this case study is the portfolio that has the highest Sharpe ratio.

Visualize the Optimal Portfolio & Portfolio Scenarios

Visualize the portfolio scenarios by using a scatter chart. Plot the volatility and returns arrays on each axis to see the relationship between risk and reward.

As a final step, visualize where the optimal portfolio appears among all of the scenarios.